Rhapsody Best Practices

Rhapsody best practice guidelines are provided to help you to develop your own best practices to guide users in designing and implementing Rhapsody configurations in a standardized, efficient, and secure manner.

For details on Rhapsody best practices, refer to:

- Naming Conventions.
- Route Design Considerations.
- Security Best Practices.
- Virtualization Best Practices.
- Maintenance Guidelines.
- Upgrading Checklist.

The guidance provided within Rhapsody product documentation is not intended to be prescriptive as there may be situations where a given solution is required to deviate from the recommended approach to fulfill the solution requirements. Where site-specific best practice guidance exists, then that guidance would take precedence.

By adhering to best practice guidelines wherever possible, you can ensure that your solutions are:

- Error-free (configuration should implement the specification correctly).
- Maintainable (component purpose and function should be clear).
- Able to guarantee that all messages should reach the correct destination in the correct form.
- Able to guarantee that messages that invoke an exception in processing are identifiable and processed appropriately.
- Able to guarantee that messages are processed through the engine in a timely manner (as required by the system specification).
- Documented consistently.
- Aligned with the intended use of Rhapsody.

Best practice guidance is subject to change over time due to:

- New or improved functionality within Rhapsody or its underlying JVM architecture.

- Improved engine and component efficiency.
- Newly-adopted and field-tested patterns that better address solutions to customer requirements, or which have been adopted as a means of resolving identified issues from past practices.

Naming Conventions

Rhapsody recommends you use a naming convention that facilitates the implementation of configurations that are essentially self-documenting, and enables components to be labeled in a manner that is clear and indicative of purpose. This allows for the grouping of components and simplifies searches across components in Rhapsody applications, thereby streamlining the development and monitoring processes.

Naming communication points and filters by including information such as their type and purpose is especially important when the components are viewed in the Management Console where they are not distinguishable by icon (unlike in Rhapsody IDE, which provides unique icons for every Rhapsody object and component type).

Furthermore, consistent naming simplifies maintenance by both internal and external personnel as it reduces the time needed to understand the purpose of a configuration.

The following recommendations provide a starting point for developing your own Rhapsody naming conventions.

General Considerations

Special Characters in Names

Components may require medium-length strings of multiple tokens (including prefixes and suffixes) to indicate use and purpose as described below. Generally, it is recommended to separate these by either the space character or the underscore character. The underscore is suitable for shorter token sequences. For longer sequences, particularly when naming items that appear within the Route Editor canvas (such as communication points and filters), the use of the space character is recommended.

- Use of the underscore in component names tends to stretch the component name horizontally and can make efficient use of layout in the Route Editor canvas difficult.
- Use of underscores in filenames such as Message Definition and Mapper files has no impact on layout within Rhapsody IDE.
- You cannot use the following characters: {}[]<>:;$\/*?"|.

Long Names

- Avoid component names which wrap to more than three or four lines in the Route Editor canvas.
- Avoid route names which are long, or nesting folders too deeply as this makes it difficult to see the entire component name without the need to scroll within the Rhapsody IDE Workspace.

Naming Format

Format: **prefix_body_suffix**

Element	Description
prefix_	The prefix typically indicates the type of Rhapsody objects, particularly components (for example, tcp_ for TCP communication points). The Management Console uses the same icon for all filters and communication points, and does not indicate their component type in every instance. • CP. - communication point. • RT. - route. • WS. - web service. The convention for component type prefixes should be outlined and discussed together with the users who are responsible for the day-to-day maintenance of the system. They may need to make alterations or additions to maintain a good naming standard.
body	The body usually describes the function of the Rhapsody object.
_suffix	The suffix provides supplementary information if required.

Lockers

Format: **function**

Examples:

- Core
- Complex Conductor Services

Element	Description
function	Lockers are named based on the requirement for the locker (for example to segregate by the role of the interfaces within the locker, or the separation requirement such as a multi-agency tenancy which would have a folder per tenant). • <Cluster> • <Tenant> • <Folder type>

Folders

Format: **##_type**

Examples:

- webPAS
- 01_Get National Identifiers
- 02_Prepare MyHR Upload

Element	Description
##_	An optional prefix that indicates the sequence number representing the different stages of processing.
type	Folders could be named according to one of the following conventions: • The primary source or destination systems they integrate with. • The primary function the routes perform.

Routes

Format: **RT.##_body_type**

Examples:

- RT.01_Input ADT from PAS
- RT.02_Transform to CMM

- RT.03_EMPI SBR Update
- RT.01_ADT Validation_P
- RT.01_EMPI SBR_OR
- RT.03_PAS ADT to LIS_OUT
- RT.03_Terminology_IN

Element	Description
##_	An optional prefix that indicates the two-digit sequence ID of the route. When configurations break a complex processing sequence across several route stages, including the sequence ID in the route name may assist in understanding the processing flow.
function	For the body make it clear what the purpose of the component is.
_type	An optional suffix that indicates the type of the route in large configurations: • _P - processing. • _OR - orchestration. • _OUT - outbound. • _IN - inbound. All direction specifications should be Rhapsody-centric; that is, inbound means Rhapsody is receiving the message while outbound means Rhapsody is sending the message.

Communication Points

Format: **CP.type_function_mode**

Examples:

- CP.tcp_PAS_I
- CP.email_Abnormal_O
- CP.ws_Register Doc_IO

- CP.ws_Terminology_OI
- CP.email_Ascribe_E
- CP.dir_Reportable_BI

Element	Description
CP.type_	A prefix that identifies the type of communication point, for example: • CP.db_ - Database communication point. • CP.dir_ - DIrectory communication point. • CP.email_ - Email Client communication point. • CP.tcp_ - a TCP communication point. • CP.ws_ - a web services communication point.
function	A body that describes the system and its purpose, for example: tcp_PAS_ADT.
_mode	A suffix that identifies the flow of messages (correlates with a communication point's mode): • _I - In. • _O - Out. • _IO - In->Out. • _OI - Out->In. • _BI - Bidirectional. • _E - Error out.

Filters

Format: **type_function_route**

Example:

- js_Process CDA Referral

Element	Description
type_	A prefix identifies the type of filter, for example: • js_ - JavaScript filter. • nop_ - No-operation filter. • map_ - Mapper filter.
function	A body that describes the function of the filter.
_route	An optional suffix used for No-operation filters used as bookends to connect routes: • _FROM_precedingRoute - the input No-operation filter with their preceding route names. • _TO_nextRoute - the output No-operation filter with the name of the next route. For example: • nop_TO_RT PAS to G2 • nop_TO_RT PAS to G2

Filter Tests

Filter tests and test messages that are used during filter testing should be saved with a meaningful name and a concise description of the test. This enables users carrying out peer reviews to use saved test messages for the review and testing of specific use cases.

Examples:

Name	Description
Patient ID Found	When the Patient ID Property is extracted and set from the database

Name	Description
Patient ID Not Found	When the Patient ID Property is not set from the database

Conditional / JavaScript Connectors

Format: **condition**

Examples:

- is Referral Message
- is Required Msg Type
- is Error Condition ADT Only

Element	Description
condition	Conditional connectors should be named succinctly to indicate their purpose. For unnamed conditional connectors, their actual condition is displayed in truncated form on the connection.

Message Definitions

Mapping Definition Files

Format: **source_to_target.mdf**

Example:

- Cerner_ADT_HL7v23_to_CareFusion_ADT_HL7v22.mdf

Element	Description
source	A prefix that indicates the source and source format being mapped from.
target	A suffix that indicates the target and target format being mapped to.

EDI Definitions

Format: **system_HL7Version_messageTypes.s3d**

Examples:

- ACTPAS_PatientMerge_HL72.3.1_ADT040.s3d
- Cerner_HL7v23_ADT.s3d

Element	Description
system_	A prefix that indicates the source or target system.
HL7Version	A body that indications the HL7 version the definition defines.
_messageTypes	A suffix that indicates the message types the definition defines.

Message Properties

Format: **valueType**

Examples:

- sourceFacilityCode
- patientRegDB_patientID

Element	Description
valueType	Solution properties and Transient properties (only relevant to the set of route stages that populate and use the property) should be named appropriately to indicate the value it stores. In some cases, properties populated during route processing may require a prefix to aid in identifying their purpose and the solution that delivered them. This is especially true of properties that are populated by a publishing route and passed to any subscribers who require them for downstream processing (typically these properties should be managed within the configuration documentation to support easy identification and use by new subscribers).

Rhapsody Variables

Format: **externalSystem_componentType_function**

Examples:

- ACTPAS_WS_UserID
- SMT_OutputDocType
- CDR_DB_Password
- RIS_TCP_Port

Element	Description
externalSystem_	An optional prefix that indicates the external system the variable is related to.
componentType	An optional body which indicates the communication point or filter type that the Rhapsody variable is being used to configure, for example: • TCP - TCP. • DB - database. • WS - web service.
_function	A suffix that indicates what the variable is used as a substitute for (for example a component's configuration property).

Route Design Considerations

Configuration best practices entail route design considerations. Some important route design considerations are as follows:

- Every filter that modifies the message body results in a write to the disk. The new message body which is the output of the filter is written to disk.
- Every filter that modifies the properties of a message (adds, deletes, or modifies the existing value of a message property) results in a write to the disk. All of the message properties of the message are written to the disk with their current value even when only some of them are modified.
- When the message body is modified, the amount of data written to disk is proportional to the size of the message body.
- The amount of data written to disk is proportional to the total size of the message property values. If more data is stored in message properties, then more data will be written to disk every time the message is modified.
- Every additional filter the message has to pass through adds a performance cost, even if the filter does not modify the message or properties. This includes No-operation filters.
- The size of the configuration increases with additional routes, filters, communication points, and message definitions. The total size of the configuration impacts the performance of Rhapsody IDE, the engine, and the Management Console.

There is typically a trade-off between the performance of a configuration and its readability or modularity. The recommendations in this topic attempt to strike a balance between the two.

Naming Components in Routes

Components should be named per function and purpose. Activities which would benefit from a sound naming convention include:

- Searching for a component in the Management Console.
- Applying a hierarchy to components displayed within Rhapsody IDE.
- Analyzing and understanding the processing logic.
- Streamlining onboarding and knowledge transfer to new team members assigned to a given project and for ongoing support activities.

Refer to Naming Conventions for details.

Organizing Folders

Folders group a set of related components. They are utilized in Rhapsody IDE (and Management Console) to provide structure and organization to the configuration. Use of a structure and meaningful names is important for the following reasons:

- To mitigate long route and communication point names and provide an easily searchable structure.
- As a high-level indication of function/purpose of routes held under the folder (for example, a folder might be used to group processing logic for input from a specific host; the host system type might be included in the folder name).
- As an indicator of a source and/or target system for messaging for the child routes (for example PAS_ADT or PAS_to_RIS).

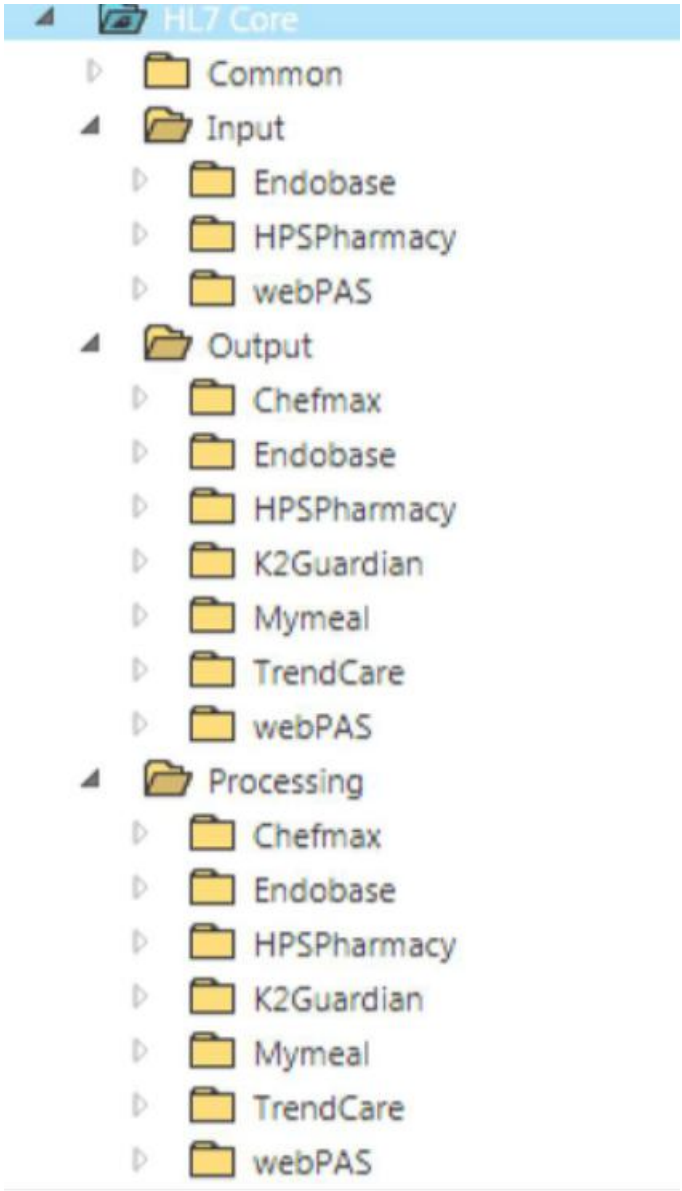
The folder structure will be aligned with the requirements of a solution. There are two folder structure paradigms that are most commonly used: SOA/Decoupled Paradigm and Interface/Solution Paradigm. However, a degree of flexibility in the folder structure may be required in order to ensure alignment with customer solution requirements.

SOA/Decoupled Paradigm

Under the SOA/Decoupled Paradigm, folders are organized under the respective locker by system name (for example, PAS, LIS). Each folder typically has subfolders that allow grouping of routes and related configurable components such as:

- Input.
- Processing (such as Normalisation, EMPI enrichment, identifier resolution, common code translation).
- Output.
- Common (for components that are shared for some or all of the above such as the Dynamic Router or Web Service Client communication points).

Folders which are not required can be omitted.



Advantages:

- Already developed components are easily identifiable based on the source, destination, or any processing requirements. This makes it easy to identify common components (such as the input part of an interface) that can be reused to meet the requirements of a new interface requiring the same data and/or processing to be output to a new system via a new interface.
- This supports decoupling of systems participating in an end-to-end interface as Rhapsody is the mediator and source or destination may be replaced with change being restricted to the set of components targeted for the new system.

Disadvantages:

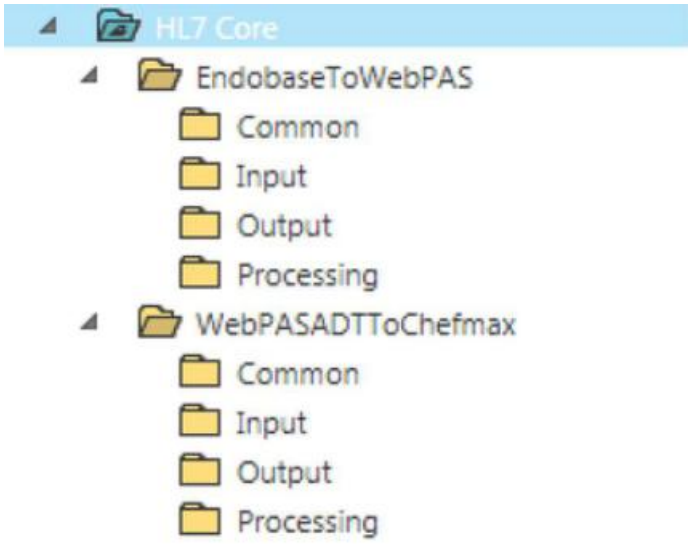
- It would not be clear to developers using Rhapsody IDE or administrators performing support actions what components are contributing to an end-to-end interface.
- Management Console users would typically have to start from the input to be able to identify other components that may be related to the interface or problem at hand.
- Additional auxiliary documentation is required over and above what is available within in-route and support note documentation to capture the relationship of the components which together are required for an end-to-end interface – this documentation may not be available within the inbuilt route notes and support notes features.

Interface/Solution Paradigm

Under the Interface/Solution Paradigm, folders are organized based on the interface or solution information (for example OrionReferralNotificationsToGP, PasADTToLab, PasToEMPI, AllDataToCDR, EMPIToSystems, ToMyHR). Within the parent folder, the following subfolders are typically used:

- Input.
- Processing (such as Normalisation, EMPI enrichment, identifier resolution, common code translation).
- Output.
- Common (for components that are shared for some or all of the above such as the Dynamic Router or Web Service Client communication points).

The folders are always present. If they are not required or, more commonly, reused from other interfaces, they can be left empty.



Advantages:

- The interface/solution purpose of the components is grouped by the parent folder name. This relationship is obvious to users of both the IDE and the management console and streamlines the process of searching for related components within the management console for support purposes. Generally, the link to the reusable components (which folder path they are in) can be captured in the newly deployed components and this combined with sensible parent folder names reduces overhead in day-to-day maintenance and management.

Disadvantages:

- Reusable components are not as readily identified as the SOA/Decoupled Paradigm – even though they may still be loosely coupled, they are deployed within the first interface that required them; future interfaces will need to rely on sensible naming of parent folders and auxiliary documentation to be able to identify if there are one or more components available for reuse in a new interface (this is generally not a major issue).

Organizing Lockers

Lockers are used for compartmentalizing the configuration in a way that allows access to that configuration to be controlled. Like a folder, a locker can be used to organize and group components, but a locker also has tools to restrict access (through the Managing Users component) so that only certain users can see and edit the configuration of a given locker, and so that only certain users can view the messages that are processed by the components in the locker.

Best practice guidance with respect to this is to determine the locker requirement on a project-by-project basis – lockers can be added as required to:

- Segregate components to ensure team members are not able to modify components that are not related to their solution or tenancy.
- Ensure that administrative users are only able to access a subset of message data and components within the Management Console (for example, to restrict access to patient data only from their site, or to ensure they are only able to start/stop components under their jurisdiction).

Messages may be directed from one Locker to another however they are only able to be distributed between lockers via the use of Dynamic Router communication points.

Segregating your configuration into lockers is not recommended unless there is a driving requirement to do so, as it increases the maintenance effort. Usage of multiple lockers requires the following Rhapsody objects and related configuration to be within the same locker for messages to flow correctly:

- Communication points.
- Message definition.
- Message tracking scheme.
- Chained/linked route.

Accordingly, they would need to be duplicated into different lockers for message flows across lockers. This results in additional efforts to maintain the components in multiple locations (for example, to add or change a message type in a definition).

In-Route Documentation

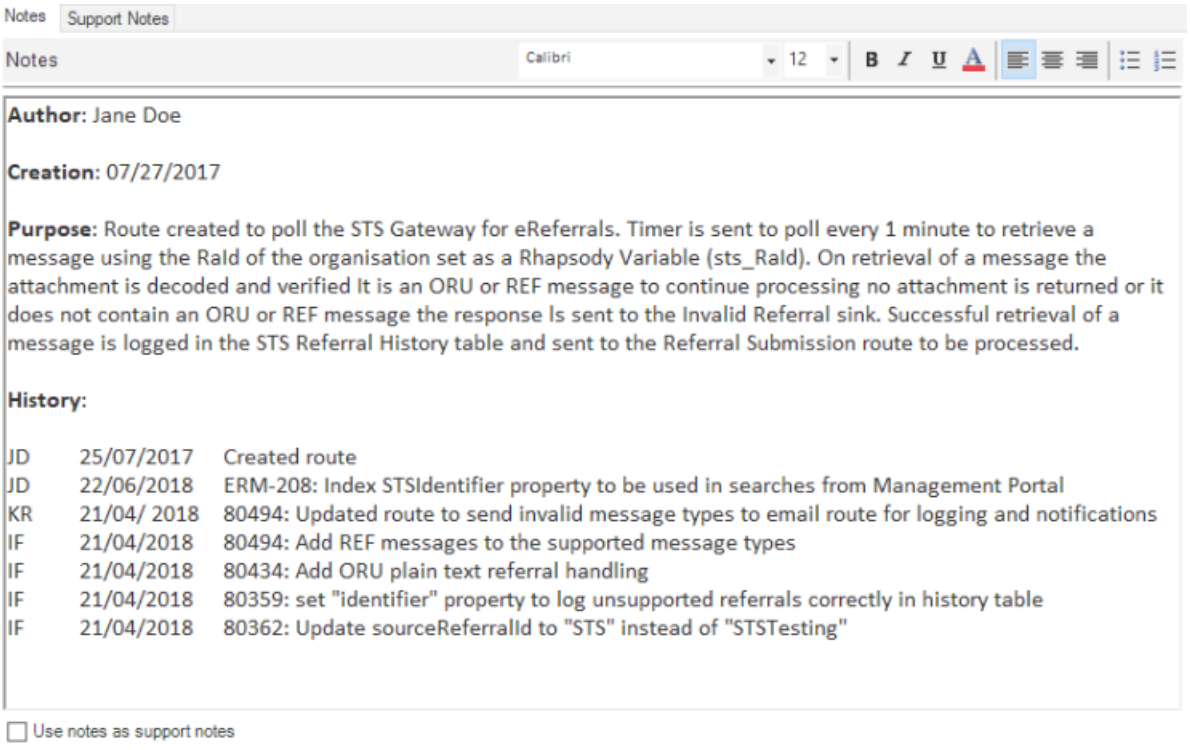
Notes

All routes, filters, and communication points, where appropriate, should contain the following minimum documentation within their Notes.

Item	Purpose
Author	The user who initially configured the component.
Creation	The date the component was created or copied.
Purpose	The description of the purpose of the component that explains what the component's intended use is and a summary of how that is achieved.

Item	Purpose
History	Modification history of the component after development (for example, BAU, fixed issue found in UAT). One line per change, each line contains the initials or name of the person making the change, the date, and the change description (include feature/bug ticket reference if available).

For example:



Comprehensive documentation of your configuration aids ongoing maintenance and is also provided in the Rhapsody self-generated documentation. For shared JavaScript library functions, JavaScript filters, and Mapper filters this will be included at the start of the code and a summary will be included within the Notes section.

Check-in Comments

For environments that are shared across multiple team members, ensure suitable comments are entered when checking in changes. It can be beneficial to follow a standard format agreed for the project. For example:

```
Description of your changes
Issue: CPO-123456
Reviewer: Jane Doe
```

Route Design

Discard Before Processing

In general, conditional filtering of messages should occur as early as possible in the processing pipeline to ensure that additional processing of the message is not incurred (particularly expensive operations such as transformation, output mapping, JavaScript filter operations). There are various methods to do so, for example:

- Conditional connector.
- Conditional logic within the filters that support it (for example, the JavaScript filter).

Fail-safe, Fail-early

Do not assume that data sent to a route will always match the specified format. Always ensure that if a given route requires data in a certain format, it is verified within that route before further processing and invalid data is handled specifically (for example, discarded via the No Match connector or passed to an error handling process as may be required depending on the route function).

Error detection should occur as early as practical so that messages are trapped or discarded before additional processing steps are incurred. For example, if a given type of message will always be discarded for a given output ensure that this is detected in the early stages of the output sequence rather than in the later stages.

Intra-Route Error Handling and Exception Processing

Error handling covers a wide range of topics, but the goal is to create a configuration where errors are the exception. Error handling should be actively planned and included as a part of the interface design. Route Error Handlers should be added to each route to handle edge cases/exceptions but it can also be used to direct known errors to appropriate destinations such as email notifications or a sink. The goal should be to minimize the number of errors that go to the Error Queue.

If the Error Queue is left to grow over time without careful management, then it will reduce the data volume available for the live message store. In severe cases, this can lead to disk space for available data running out earlier than calculated or expected.

In-filter processing can also be used to detect certain error conditions (missing information, logical error in a combination of fields, change of the input message type

compared to a known structure, a message that is not valid based on business rules rather than format/definition). The JavaScript filter can be used to call the addErrors() method in these types of scenarios to add specific information about the error and then cause the message to be output to the error connector/path of the route

The method for handling the errors will be determined based on the interface pattern and requirements. However, in all cases, Error Handling should be designed and implemented (as per pattern and customer requirements) so that it is adopted from initial configuration forwards rather than added on at the end.

Options for handling errors include:

- Unhandled - the message is sent to the Error Queue (this should be the last resort, not the norm).
- Direct handling - logic is implemented to manage messages with known errors; the error detail is available from the message object by calling the getErrors() method in a JavaScript filter. Once processing is complete, the message may then be:
    - Ignored, as this is a known error which does not require further processing and output to a Sink.
    - Handled explicitly by output to a Communication Point/Route for this purpose (for example, to automate raising a helpdesk ticket or to email specific people with the event details).
    - Re-routed to the Error Queue.

Errors relating to filter failure (rather than processing of the message content) bypass the Error Output connection and are placed directly into the Error Queue. For example, system-level errors will result in this behavior.

Route Layout and Processing Complexity

In essence, the route design canvas of the IDE presents a map or a flowchart of the high-level logic required to process messages. The following guidelines are recommended to ensure that the flow of data through a route can be easily interpreted and checked to ensure that the route logic is consistent with the requirements as well as for streamlining support and maintenance:

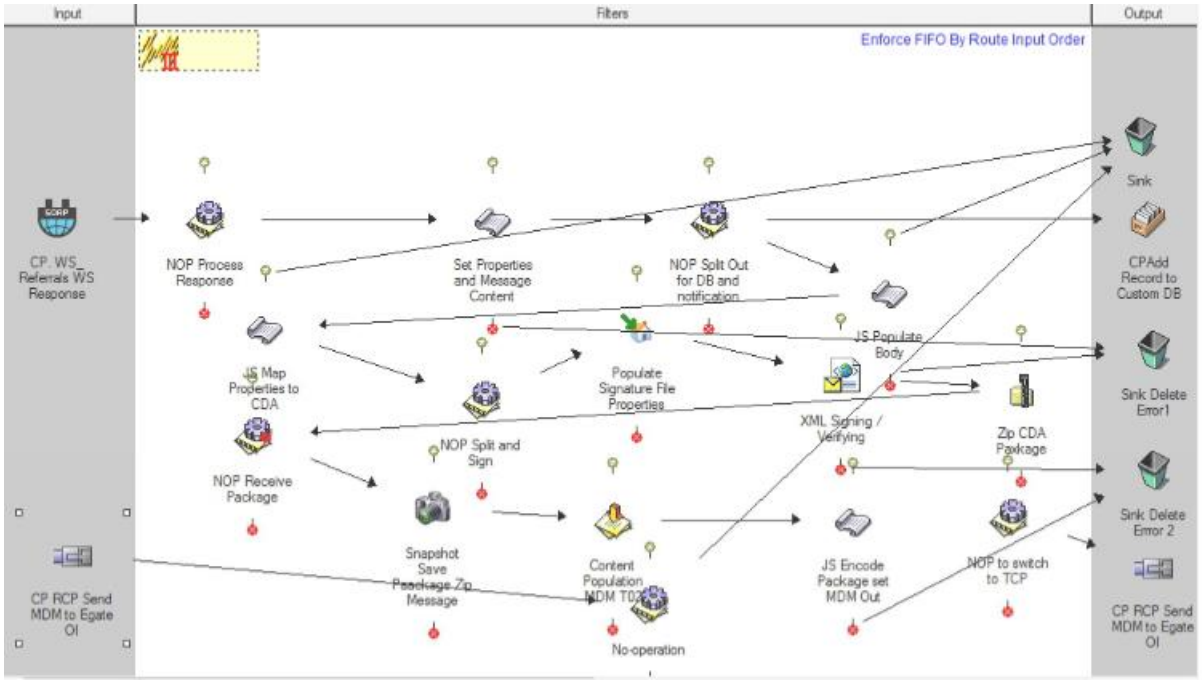
- Message path flows generally from left to right (input to output).
- For some scenarios, the route may output then return to an input path such as when processing a message and then the Acknowledgement via an Out->In

communication point. This is a normal scenario and the flow is still generalized as left to right, similar to the operation of a cursor on a page of typing.

- Input connections should be made from a component to the left of the filter.
- Output connections should be made to components to the right of the filter (this also includes No-match and Error connection points).
- No-match connections should generally be made above and to the right of the filter.
- Error connections should generally be made below and to the right of the filter.
- Note that the use of the Route Error Handler may streamline error processing and avoid requiring individual paths from Error Connectors, as well as No-Match Connectors in some scenarios (if a condition is not met and there is no path from the No-Match the message will route to the error path).
- Connector paths do not cross each other (this can often be achieved through the use of additional No Operation Filters).
- Avoid loops in the configuration that bring the message back to an earlier point in the same route.

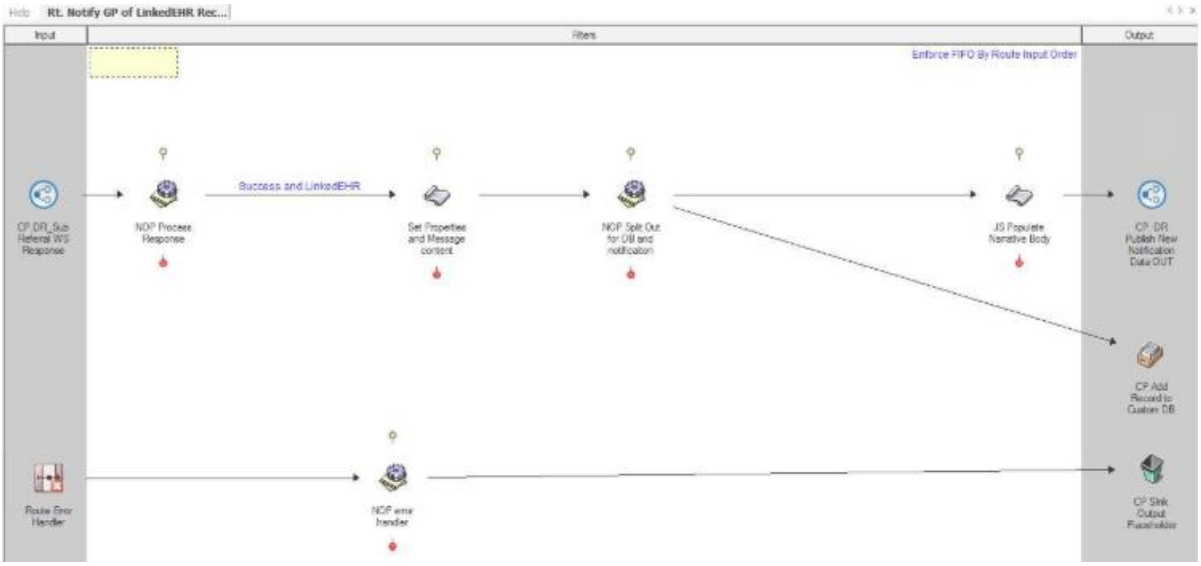
Overly complex routes with multiple significant branching and/or return to filter components generally indicate that the configuration is attempting to achieve too much in a single route. In this case, consider splitting the single route into several route stages, each of which performs a specific function before passing the message to the next stage for processing. This can be achieved either using Dynamic Routers or through Linking/Chaining Routes.

The following screenshot shows an example of a monolithic route that would be better deployed as a series of route stages to simplify the overall layout and avoid potential logic issues:

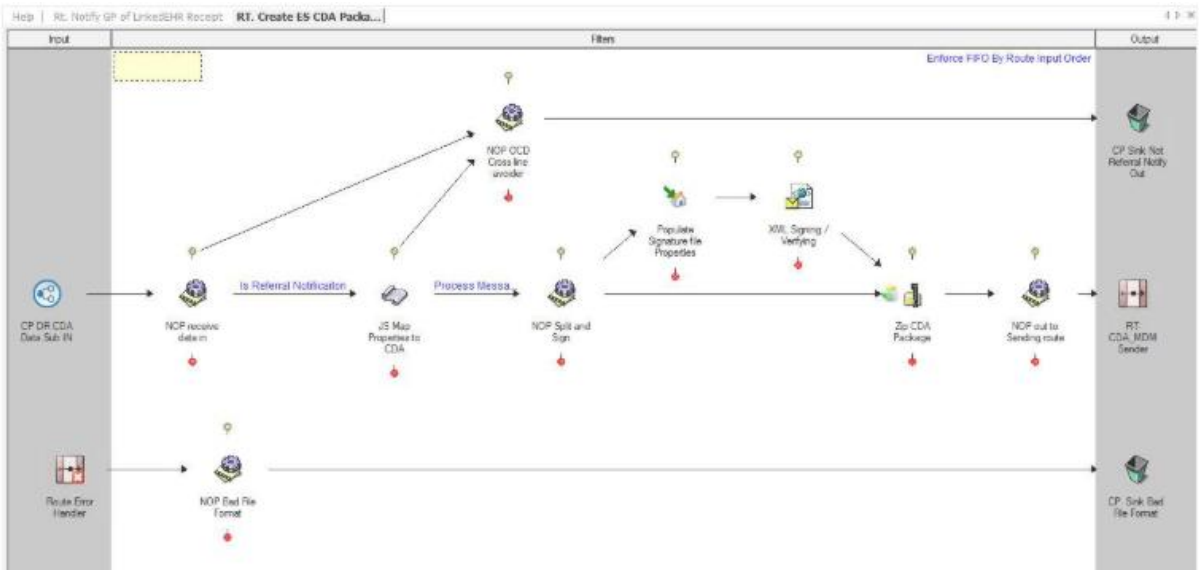


The following screenshots show an example of simplifying layout and targeting smaller, more specific, areas of functionality using several route stages.

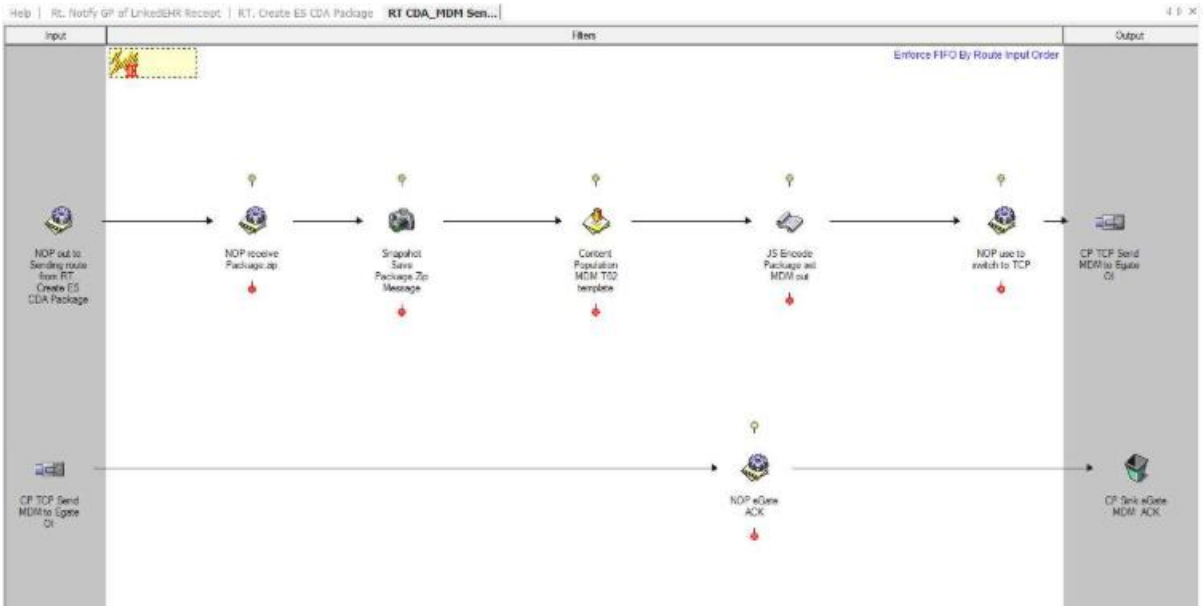
A Simplified Route Layout - Stage 1:



A Simplified Route Layout - Stage 2:



A Simplified Route Layout - Stage 3:



Linking/Chaining Routes

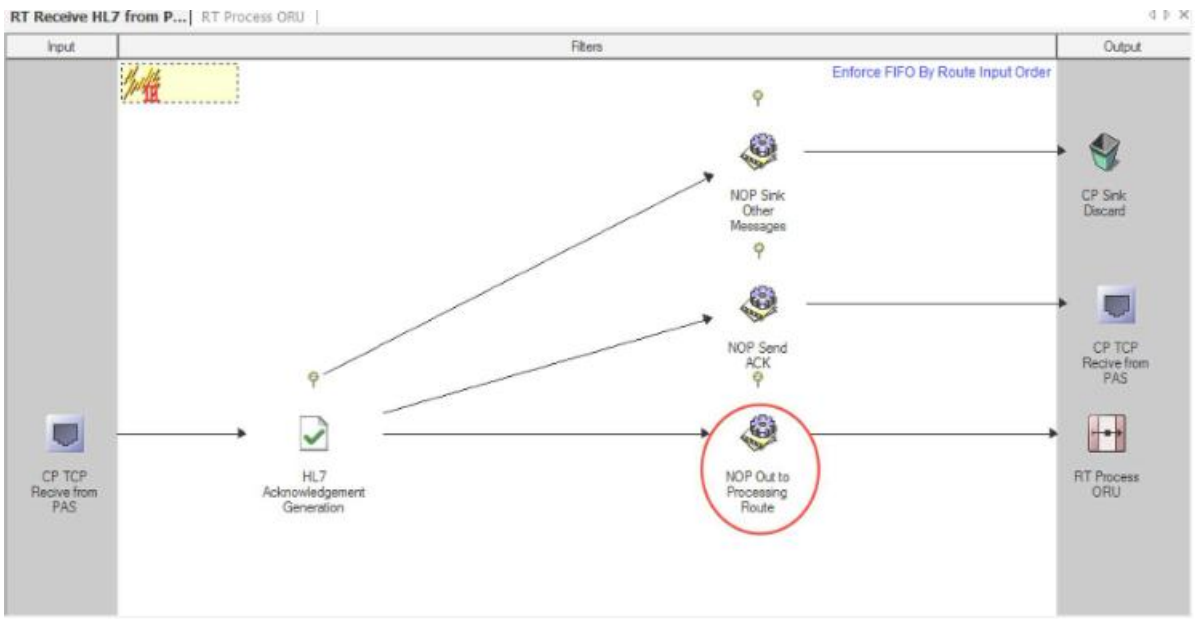
In a complex configuration, it is often useful to distribute the processing logic across multiple routes, limiting each route to one high-level function and ensuring that the clarity and purpose of each route are maintained.

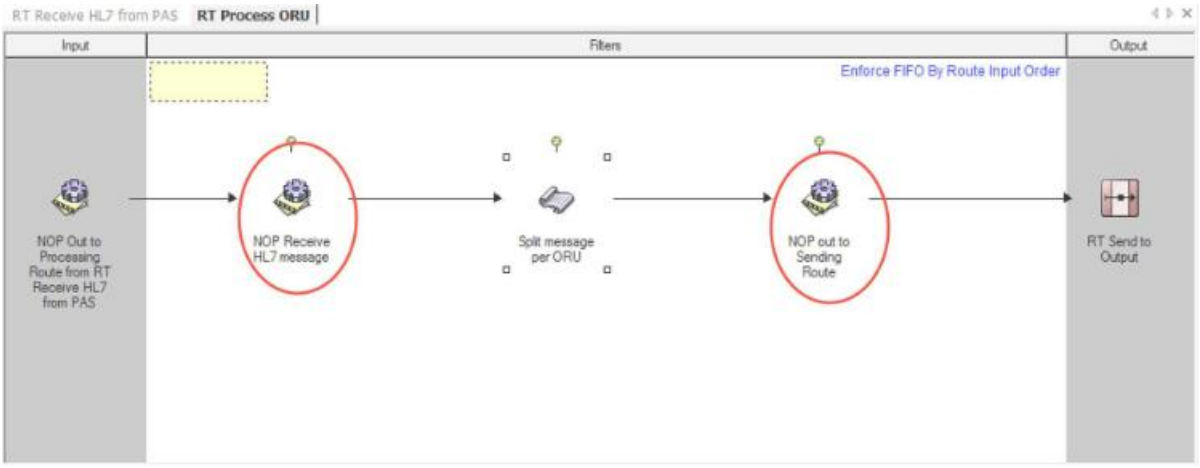
In some scenarios, it may be beneficial to directly link the routes in a chain (this is generally the case where the overall requirement cannot be achieved unless all the links in the chain execute in sequence). For other cases, the use of dynamic routers would be recommended to separate the stages of the route processing.

When routes are chained there is a risk that re-arrangement of filters or changes to message paths may lead to disconnection of the link. For this reason, when chaining routes together follow the following process to ensure the connections are managed properly and can easily be identified:

- Place a No-operation filter as the last processing element in the upstream route.
- Place a No-operation filter as the first processing element in the downstream route.
- Drag the downstream route (selected from the configuration workspace) onto the output side of the upstream route.
- Connect the No-operation filter on the upstream route to the route icon in the output frame. This will place the connection into the correct location on the paired route.
- Connect the No-operation filter on the downstream route to the connection icon in the Input frame.
- Check in both routes to save the configuration.

This model is often described as providing bookends to the routes, and generally protects the connections between the routes. It is, however, prudent to confirm the connectivity between the routes if changes are made to either route.





Multiple input connections may be made to the output route icon in the upstream route. Each will result in a separate input connection to the downstream route, thereby allowing separation of processing for each path.

Message Definitions and Parsing

If a message definition is placed on the route, message parsing will occur prior to processing by the first filter on the route. This ensures that properties defined at route level are available to condition connectors and certain filters.

Message parsing is required by the following communication points and filters:

- HL7 Acknowledgement Generation filter.
- EDI Message Validation filter.
- Content Population filter.
- Code Validation filter.
- Date Validation filter.
- Mapper filter.
- Code Translation filter.
- JavaScript filter (if the message has not previously been parsed)
- XSD Validator filter.
- HIPAA filters.
- Database filters and communication points that use message fields in the query.

As message parsing incurs a processing cost, it should only be undertaken when necessary. Use the following guidelines to avoid unnecessary re-parsing of messages:

1. Place only one message definition on a route if parsing is required for the route to operate.
2. A message definition is not required for the following routes:
    a. Pass-through routes.
    b. Routes with messages that are processed purely using the following methods:
        i. String-based processing.
        ii. XML processing via E4X.
        iii. XPath processing of well-formed XML.
        iv. HAPI message processing (as HAPI does not require a message definition).
        v. Using only the HL7 Message Modifier filter.
        vi. JSON processing.
        vii. Certain scenarios using Web Services or Database Orchestration
3. Avoid using multiple filters which require message parsing when possible:
    a. JavaScript filters cause a message parse operation for each filter – where possible combine multiple JavaScript filters into a single filter (the use of shared JavaScript libraries can be used to offset the overall complexity and maintenance effort for single JavaScript filters. Refer to JavaScript Use and Features).
    b. Message Validation filters on a route with JavaScript or Mapper filters – often the validation component can be achieved within the other filter, allowing removal of a parse operation.
    c. Property Population filters which map message fields to properties on a route with JavaScript or Mapper filters – again, these filters can typically achieve all the required functionality
4. Avoid adding multiple message definitions to a single route. Rhapsody does not guarantee which message definition will be used in this scenario. The first definition which allows a successful parse may not necessarily be the expected one, which can lead to inconsistent results. Typically, if a pattern looks like more than one definition is required on a route to achieve the outcome, then it is probable that:

a. The solution is performing a mapping operation that is better suited to the Mapper.

b. The route should be split into multiple stage routes each targeting a specific area of functionality and only requiring at most a single definition per stage.

Multi-threading versus Single-threading

Rhapsody is a multi-threaded Java application and uses 10 worker threads (by default) to manage all the route processing. In general, the processing cycle typically has the following states:

1. Route worker thread becomes available.
2. Next route operation selected from the queue (each operation equates to processing one message through a single filter).
3. Processing transaction starts.
4. Message processed by the filter logic.
5. Processing transaction completes.
6. Route worker thread becomes available.

This design enables multiple messages on a route to complete processing by a filter at more or less the same time, unless either filter concurrency restricts processing or the route is constrained to processing in strict message order by FIFO constraints.

The filter concurrency is normally set to zero which allows the multiple and potentially out of order processing (note however when FIFO is set on a route even though the messages may be processed out of order due to different paths or complexity of messages in a given route, the messages will always leave the route in the FIFO order received). It may be necessary to limit concurrency in one of the following cases:

- Message processing order is significant - messages must process in the order of receipt (particularly if there is an in-route orchestration and the processing result of a later message would be different if the earlier message has not processed).
- Slow filter processing - in these cases, under moderate to high loads, it may be possible for all of the worker threads to be processing messages through an instance of the same filter at the same time which will effectively block other processing.

Filter Concurrency is available as a filter property for filter types likely to experience this issue. Setting the concurrency to a non-zero value limits the number of messages able

to be processed through the filter instance to that value. The Database and Execute Process filters are particularly prone to needing attention to concurrency.

Dynamic Routers

Use of the Dynamic Router communication point (dynamic router) is recommended for decoupling routes which publish and distribute data, process data or subscribe to, or receive and send data to downstream systems. Earlier versions of Rhapsody were constrained to the use of chained routes to achieve the same outcome. Use of dynamic routers at the boundaries of internal routes is advantageous because new consumers of existing data can be added in the future without needing to modify existing, validated, in-production configuration, thus mitigating the risk of introducing issues or regressions to the current function.

However, because data is Published/Subscribed to in a dynamic fashion there is potential to introduce issues within configuration if the following guidance is not followed:

Subscribers (routes which use a dynamic router configured for input mode):

- A route which contains that input dynamic router should verify the data is of the required type before performing any operations. This prevents errors or exceptions as data sent to the input dynamic router could change after deployment. For example, a source system could start sending new data types via an existing input interface or a newly deployed configuration could match an existing target name of an input dynamic router.
- If present, the router:Destination property should have the current value removed (set the property value to null if it is not required, or to a specific value as required) to ensure that processing through any down-stream output dynamic routers does not result in route loops.

Publishing routes (routes which use a dynamic router configured for output mode and may use dynamic destination lists set via in route JavaScript processing):

- Use Target Name (for example, @theTarget) if possible, multiple receivers can be configured with the same target name to receive the set of data from the output dynamic router. This method is simpler to manage and less prone to error than the path method. It supports delivery to multiple components with the same target name. In addition, the path value would have to be updated if the component is moved within the IDE to a different path.
- While a single instance of a dynamic router, using dynamic destinations, can be used in multiple routes to reduce communication point clutter, there are disadvantages to this approach which need to be balanced for a given solution's requirements:

    - The Archive queue will contain data sets from the multiple routes – this may obfuscate the cause of issues or increase overhead for some Management Console administrative tasks.
    - If the communication point must be shut down for any reason it will block the output (and result in queuing) for all routes that this output component is contained in.

- Where destinations are set dynamically, care must be taken if the list of destinations being set within the JavaScript filter is populated from a source which can change (for example, a lookup table). If a target name does not exist, the message will be routed as per the configuration setting for **On Missing Dynamic Destination**.
    - We recommend immediately adding an input dynamic router with a matching Target Name and checking in the component whenever making changes to dynamically generated destination lists.
    - Where retiring a component, remove the destination from the generated property list first then retire the matching input dynamic router component/route

Identifying Bottlenecks in the Route

When you create a route, it is recommended you identify the bottlenecks in order to optimize performance. In the Management Console, there are a number of reporting options available that can help with this. The performance statistics report can be used to identify which filters in the route take the most time to process the message. However, as there is a processing overhead when running a performance statistics report, it is recommended running it in a non-production environment.

Queue Depth

Communication points (depending on type) have an input and/or an output queue (depending on their directionality) while routes have a processing queue. In general, messages flow through the Rhapsody engine more or less unhindered. Messages may queue on some components at some stages because of load, slow processing, or some external factors, but queuing is generally transient.

Factors which can impact the size of queues include:

- A requirement to throttle messages or only send messages to downstream systems during certain hours
- An issue at the receiving system end that will result in a backlog within Rhapsody until the issue is resolved

- A system which sends messages in large batches or bursts (either due to the nature of the information exchange or because it has been offline and needs to send a large amount of data that would normally flow over a longer period at a lower rate of messages per minute)
- A system is decommissioned while messages are still being processed within Rhapsody
- Resource intensive message processing and/or very large message processing within a route
- Single threaded/Strict FIFO requirements

Attention should be paid during route development to the potential for messages to queue (particularly with respect to the last two list items above which are controlled by Rhapsody). To reduce the potential for queuing:

- Develop a solution using standard product-supplied Rhapsody communication points and filters in preference to developing custom components. This reuses code libraries as typically the product-supplied components align with, and are tested with respect to, Rhapsody performance and multi-threading capabilities (which in turn generally avoid large processing queues in most scenarios).
- Constrain the use of Strict FIFO or interfaces with filter processing that requires no concurrency to only those use cases which absolutely require it to ensure minimal impact to engine performance and queue depth.
- Ensure components are correctly configured with Auto Start parameters to ensure queues do not form because one of the components has not started.

There are several significant impacts of queuing on the engine:

- Queues are managed as memory objects; consequently, large queues will lock memory for the queue and make it unavailable for normal processing.
- When an engine restarts, the active objects are validated before processing can resume; consequently, large queues will take a finite time to validate and delay the resumption of normal processing by the engine.
- Messages on queues are not eligible for removal from the datastore by the Rhapsody Archive Cleanup process (for the scenario of a decommissioned system the messages may need to be manually deleted from the queue so they are eligible for clean-up).

Communication Point Considerations

- Wherever possible use an instance of a communication point on a single route only. This provides advantages in terms of storage, queue management, and message tracking.
    - There may be limitations and considerations for a specific solution (such as the licensed number of communication points) that need to be considered.
    - In some cases, it is advantageous to reuse a communication point on multiple routes to reduce maintenance overhead and effort for those communication points with significant configuration components – web service clients connecting to the same service with a number of different methods are an example of this. Database communication points used to log/store all received messages may be another.
- Use multiple sinks rather than a single, general purpose sink. This allows you to see how many messages are being discarded for a particular system.
    - Communication points such as sinks and dynamic routers where messages do not enter or leave the engine instance do not count towards the licensed number of communication points.
- De-batch large messages in a Directory communication point instead of a de-batching filter (either the Batch/De-batch or Zip/Unzip filter). This is more efficient as the communication point can read each message out of the batch from disk without loading the complete batch into memory.
- Ensure the **Connection Retries** property for a communication point is set appropriately for the system that it is connecting to, and has been tested to ensure it is tuned appropriately.
- Where multiple actions need to be performed on a message when it is received on a given route, it should be received on the route then split via the use of a No-operation filter for each action. Do not directly split the input from the input communication point as this results in duplicate parsing of the message and duplicate storage of the input message and properties within the Rhapsody datastore.
- Where connections between two Rhapsody instances are required, the Rhapsody Connector communication point should be used instead of the TCP communication points.

Filter Considerations

- Pay attention to earlier recommendations for combining filter functions where possible to avoid multiple message parses and increased datastore volume use.

- Only set filter concurrency to a non-zero number if performance analysis indicates the filter is consuming large amounts of resources and is negatively impacting overall system performance. Setting filter concurrency to 1 will greatly reduce the throughput (as the route will become single-threaded) and is only required in a small number of use cases (such as where absolute FIFO is required within the route, rather than simply for end-to-end message delivery).
- The use of database filters is only suited to certain scenarios, as outlined in the Database Considerations.
- Avoid using excessive No-Operation filters for route commenting – this practice introduces excessive growth of the events within the Message Store and has been demonstrated to negatively impact large-scale deployments. No-operation filters should only be used to:
    - Split message processing paths.
    - Improve route layout by avoiding crossed connectors.
    - Support Message Collection (if this feature is used, ensure that the name of the filter reflects this).

Conditional Connector Considerations

Override the Condition Description

Always override the default condition description for the connector and replace it with the shortest text that makes sense to describe the condition (for example, is Required Message Type, eReferral Only).

Rhapsody Variables in Conditional Connectors

Though Rhapsody variables can be used for any value that may require an update in future, it is not recommended they be used in Conditional Connectors for the following reasons:

- The condition they are matching is not immediately obvious when viewing the component or troubleshooting condition test results – the user must access the Variables Manager which cannot be opened at the same time as the UI component of the Conditional Connector.
- In the current Rhapsody version (and all previous versions) they are not exported between environments unless you select the **Export Unused Rhapsody Variables** option – this can result in a lot of unnecessary clutter in the downstream environments which will have to be manually mediated.

Database Considerations

Refer to Database Components Best Practice Guide for details.

JavaScript Use and Features

Coding Guidelines

The following general guidelines are the general guidelines for writing JavaScript code within Rhapsody.

Dos:

- Use shared JavaScript libraries to add functions that can be reused across JavaScript filters.
- Improve code readability and self-documentation through the longhand versions of constructs such as the ternary operator.
- Code within the JavaScript filter or the function within the JavaScript Library should follow the same documentation convention as the route notes for capturing the date, author, purpose, and modification history.

Don'ts:

- Avoid using multiple JavaScript filters in the same route. Instead try to combine into a single JavaScript filter where possible, and utilize the shared JavaScript feature to reduce filter code complexity.
- Do not use system commands, for example System.getProperty(). These calls are generally synchronized so that two calls cannot be made at one time.
- Do not use System.exit() in any JavaScript filter as this will cause processing to halt.
- Do not create delays in the code of a JavaScript filter. If a delay is needed, If a delay is needed, use Message Collection functionality in the route.
- Avoid intermingling setField and GetText calls in your code. This switches the editing models (from parsed message to message text).
- Avoid excessive commenting – code developed following the above guidelines should be readable and self-documenting, comments should be used to identify major features, executions, or considerations of the code.

Using Shared JavaScript Library Functions

We recommend using shared JavaScript library functions wherever possible as this ensures a central location for management, testing, and updates (rather than requiring each JavaScript filter that may contain a copy of the code to be updated). This, in turn, streamlines the code within shared JavaScript filters and supports scenarios where

multiple JavaScript filters can be combined into a single filter without combining large amounts of JavaScript code in the single filter.

When creating shared JavaScript library functions take the following into consideration:

- The shared JavaScript library function does not have scope visibility of the standard JavaScript objects available to the JavaScript filter (such as the Input and Output message or the log). Ensure you pass these as parameters where required.
- Shared JavaScript library functions can only access other shared functions that are contained in the same library in the current version.
- For debugging during initial development, it is often better to build and test the function within a JavaScript filter and then promote it to the shared JavaScript library (taking into consideration any scope dependencies as previously noted).

Considerations for Multiple JavaScript Filters and Parsing/Route Performance

The use of multiple JavaScript filters can have an impact on the performance as each filter requires a parse of the message, thereby resulting in multiple parses (note this is not the case if there is no message definition on the route in question). In some cases, multiple JavaScript filters may be required due to reasons such as:

- Processing logic – for example after a message has been processed by a JavaScript filter, it needs to be split because one path requires the message/properties at that state while the other path requires additional processing by a separate JavaScript filter before being output from the route.
- Separating complex JavaScript operations into discrete units of function to simplify testing and maintenance. There are two strategies for solving this issue:
    - Move the unit into multiple shared JavaScript Library functions (which are tested and validated separately, and may be maintained separately) and then call the functions from within a single JavaScript filter. This is the preferred method – particularly where the presence of a message definition on a route would otherwise cause multiple parses of the message.
    - If there is a compelling reason to use multiple JavaScript filters for a given solution's requirement to separate the processing into functional units, ensure that this method is only used on routes that do not have a message definition associated with them. This can be mitigated for routes that do have a message definition associated with them if the performance testing demonstrates no significant difference in route

transit time for the use of multiple JavaScript filters compared to a single filter.

Message Mapping Options and Considerations

General Considerations

Message mapping and transformation is supported through a number of Rhapsody features and components. Previously, the general rule was to use the Mapper filter as the compiled code was more efficient than the other available options. However, improvements in engine efficiency mean that while this is still generally true, the difference in execution time is often insignificant and several other factors and constraints must now be considered when deciding how to manage message mapping:

- The Mapper filter requires a Mapper Definition File, which in turn requires an input definition and output definition file for the message formats. The usage of the Mapper filter is generally better suited to scenarios where there is a significant difference between the input and output format and where significant effort is required in mapping between them.
- Mapping or transformation that involves a small number of fields is better suited to the JavaScript filter (which may include the use of the shared JavaScript libraries) since the transformations can be grouped as execution lines and readily identified when maintaining the solution. This is often the case when the input and output formats share a format definition. Even for cases where the definitions are different, the same requirement can be achieved for smaller transformations by either using the HAPI libraries or splitting the transformation into an input and output stage each with a separate message definition (providing that while different, the definitions are still compatible).
- Mapping HL7 to XML or vice versa can be achieved through either the use of E4X template processing within the JavaScript filter or by using the Mapper filter. Please consider the following exception scenarios:
    - When dealing with scenarios that require insertion of XML snippets into an existing XML document, these are currently best suited to delivery via mapping in JavaScript filter using E4X processing.
    - When parsing large XML files while performing mapping to and from XML, these are currently best suited to delivery via mapping in Mapper filter.
- Mapping where features required are only exposed through JavaScript Objects will require the use of the JavaScript filter, for example:
    - HAPIMessage Object.
    - JSON object.

    - Snapshot loading and processing (especially where multiple snapshots are used).

- Mapping and transformation of messages that are known to not always conform to a strict message format are generally best handled within the JavaScript filter (this is often true for like-for-like migrations of legacy interfaces where input messages may need to be processed without definitions to correct or retain format/value issues within the message) as there may be the potential to correct or ignore the error and continue processing. Use of the Mapper in this scenario will either fail or ignore depending on the **Fail on Errors** property configuration of the Mapper filter.
- Mapping via the use of the Intelligent Mapper will require the development of the mapping within an Intelligent Mapper mapping project along with the use of the Intelligent Mapper filter (this is effectively a specialized version of a JavaScript filter that only supports the use of the read-only JavaScript Mapping developed within the project).

Specific Considerations for the Mapper Filter

- Use submaps for code reuse and easy maintenance.
- Structure your maps based on the output message structure.
- When using the Automap feature of the Map Designer care must be taken. This feature is best suited to message types where the input is very similar to the output; where this is not the case some components may not be auto mapped and “To Do” sections will exist which require manual coding for the mapping. Therefore, always ensure that if the Automap is used the code is searched globally for “To Do” sections, and these are completed prior to use in the engine.

Lookup Tables

Lookup tables within Rhapsody provide more flexible options and better performance than database filters such as the Generic Code Translation filter. They are typically suited to situations where less than 600,000 rows are expected. Moreover, they may be accessed directly from within JavaScript and Mapper filters as part of message manipulation operations rather than requiring a separate component execution (communication point or filter).

Care must still be taken to ensure code which uses these features is safe. However, as they are part of engine runtime processes, they are less prone to errors as may be encountered with database filters (such as connectivity failures).

Custom Components

Custom components, which are developed using Rhapsody RDK, should only be used when the required functionality cannot be achieved using standard Rhapsody communication points and filters. The default position should be to use the Rhapsody standard components if they can achieve the outcome because:

- Custom components may not provide the same level of detail for logging and Management Console.
- Custom components typically require a different skillset to produce and maintain when compared to standard Rhapsody components.
- They are not supported by Rhapsody – any issues with custom components are the responsibility of the client to maintain and resolve.
- Reuse of existing customer libraries via the RDK can lead to issues due to:
    - Java version issues.
    - Execution of multiple functions which negates the message inspection, logging, and management aspects of Rhapsody.

Custom components may be required or considered where:

- Connectivity to a system or function is provided by a third-party software component (API, EJB, etc.) that is not directly supported by current Rhapsody components, for example calling Oracle HMPI EJB functions to retrieve a patient Single Best Record.
- The functional requirement is of significant benefit to the solution or is a key requirement for the solution but cannot be achieved by the current out-of-box components (for example development of signing and encryption filters using ADHA libraries to upload documents to My Health Record).

When developing a custom component follow these processes:

- Raise a Jira ticket for development review of the proposed component development to ensure that:
    - Similar functionality is not on the roadmap.
    - The proposal is sanity checked.
- If the feature is beneficial to multiple customers or the product, there is an opportunity to uplift the component to become a standard feature of the product
- Ensure the target functionality of the component is specific - the component should not generally encapsulate multiple unrelated functions (unless this is the requirement for the component such as may be required to manage a distributed

transaction with orchestration). This is to streamline troubleshooting and maintenance efforts.

- Ensure the logging events are in line with standard Rhapsody components so that the information presented to the user within the Management Console is consistent with other standard components.

Refer to Rhapsody Development Kit for details on how to build custom components.

Message Properties

Avoid Assigning the Message Body to a Property

Some implementation scenarios require capturing a snapshot of a message body as a property so that it can later be retrieved. In versions of Rhapsody before Rhapsody 6.2.1, message properties were used for this purpose. However, message properties are not designed to convey large amounts of data. Doing so has a negative impact on performance as well as increasing the amount of datastore used for each message operation. This can be avoided as of Rhapsody 6.2.1 by one of the following methods:

- Using the Snapshot Save filter to save the current message and its properties. This can then be used to achieve the desired outcome by:
    - Using the Snapshot Load filter for simple scenarios where you want to replace the message on the path with the previously saved snapshot.
    - Using the JavaScript Object functions to load the snapshot or load a snapshot by ID. While the rhapsody:SnapshotMessageId property contains only the most recent saved snapshot, it is possible to save and retrieve multiple snapshots using custom properties and then load them through the JavaScript loadMessageSnapshotForId method
- Saving a temporary message into a database using a Database communication point (via a stored procedure) and store the return value (a unique link to the stored message) as a message property. This can then be used by a downstream process to retrieve the saved message from the database and process it (refer to Database Considerations). This is crucial because message snapshots can only be loaded while they are present within the Rhapsody datastore (within a single cluster only), in other words, while they have yet to be removed by the Archive Cleanup process, or processed to a different cluster. In most scenarios, this is not an issue as a snapshot is not eligible for removal in the time frame where a typical sequence of route processes is executing in a single cluster. However, there can be a significant delay (from days to weeks) between the process that stores a temporary message and the process that completes the output, or where the message is sent to another functional cluster via a Rhapsody Connector.

Managing Message Properties

As they flow through the engine, messages can accumulate message properties. This can result in 'message property spam' in later routes where large numbers of message properties have to be managed and understood in downstream use. This can impact the effort required to analyze message property changes on a message path in the Management Console. Large numbers of properties also impact the size of datastore use and overall engine performance.

The following strategies are recommended for managing properties:

- Custom properties which are important for use in downstream routes deployed by a solution should be documented in a property register for easy reference by future teams maintaining or extending the solution (for example, the set of properties set by the publisher of data which can be used by subscribers to determine if the data should be processed or filtered out).
- Custom properties received by a downstream route that are no longer required should be removed after use when possible (by setting the property value to null).

As this typically requires a filter, this should be done when a given route stage would already be utilizing a filter such as the JavaScript filter which can update the properties rather than adding a filter solely for this purpose, unless the downstream properties are unmanageable in number.

If a large number of properties are required for downstream processing consider creating a JSON object to contain all the property-value pairs, this could be set as the message text and saved as a snapshot for use in downstream routes and filters rather than polluting the message properties with large numbers of properties.

Indexing and Searching

By default, Rhapsody properties are not searchable within the Management Console. Message properties can be defined as indexed to be used for message searches in the Management Console based on the content of this property, obviating the need for a full meta-search across all messages.

Properties can be indexed on a per route basis or with JavaScript calls. Properties indexed for input can only be used in input search; properties indexed for output can only be used in output search. You may, however, index a property for both input and output.

In order to prevent unnecessary processing and disk utilization by the message property indexing logic, ensure indexing is restricted solely to fields that are required for fast lookups on message searches.

Rhapsody Variables

Avoid hardcoding values of ports, IP addresses, hostnames, and other configurable parameters into your routes or communication points. These parameters should be configured through Rhapsody variables so that they are easy to access and change. Using variables exposes configuration settings in one single location that would otherwise be embedded in routes and communication points.

Use Rhapsody variables for environment-specific parameters such as:

- File paths or directories.
- Email addresses.
- Remote system host and port details.
- Local service host and port details.
- Connection details for external systems and databases such as username.
- Web service URLs.
- Access credentials.

Using Rhapsody variables in this way is also useful when migrating a configuration from a non-production environment to production. Rhapsody variables allow for differences in system configurations between environments and provide a smooth transition of configuration to the production environment as the variables are not overridden by default when Rhapsody configuration is imported from one environment to another. This allows different configuration settings to be used in production and non-production environments. The use of variables also helps expose configuration settings that would otherwise be embedded and hidden, making for quick access and change.

Testing and Peer Review

Peer review is recommended during development and is required before promotion of a solution between environments or merging into a master repository. Peer review should take into consideration the documentation (including route self-documentation, test results, and any documented deviations or design decisions which are not normally considered best practice). The reviewer is typically not the same person who develops the solution.

Development Testing

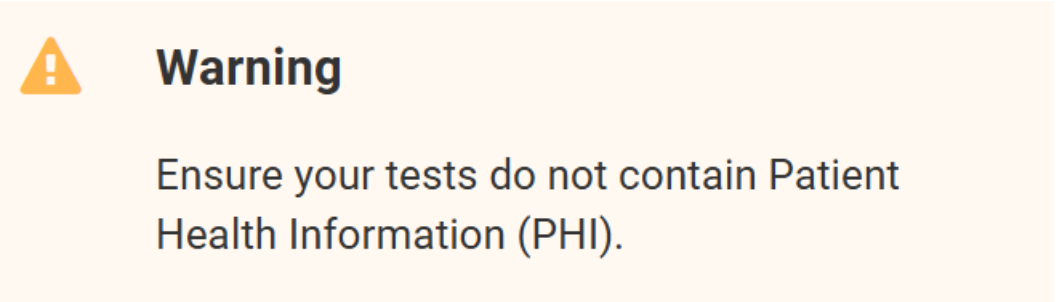
A solution will require testing and validation to demonstrate the soundness of the solution. Evidence of the completion and result of testing will typically be required by the peer review process and ultimately the customer. The level of testing and related documentation may differ by customer/contract, however evidence of completion of

the following tasks is recommended before promoting a solution from a development to production environment:

- Unit testing of filters and conditional connectors via the inbuilt test feature for:
    - Invalid cases
    - Valid cases
    - Both of which may require permutations based on the number of conditions or branches executed in a given component
- Unit testing of message flow (input, output, property, and message change throughout the route as per the message flow in the Management Console) for individual routes
    - For both valid and invalid messages, messages which demonstrate conditions of the route end to end.
- End to end message testing for the sequence of routes related to a given interface or set of interfaces for the current solution under development.

Note the following:

- In some cases, it is not effective to test certain conditions at a unit level within the filter test function – particularly if the filter requires the result of processing from earlier route components; in these cases, the testing will revert to the route level unit testing.
- Where a filter test requires a message snapshot the configured property must contain the ID of a message still within the message store if tested within the filter. This may require the use of a hold queue to ensure the identified message is available.



Security and Access Controls

Most communication points support secure transport and/or authentication protocol. Therefore, Rhapsody's default best practice recommendation is to always enable the secure transport options. Refer to TLS/SSL Support in Rhapsody for details.

In addition, where authentication is required (for example, basic authentication, WSS for SOAP, etc.), it is recommended you always opt for encrypted/hashed passwords. This mitigates the risk of plain text credentials being captured in the message log as part of the transported message.

It is also the best practice to keep the security setting identity between the production environments and at least one non-production environment.

Other General Considerations

Assign a Specific User to Run the Rhapsody Process

On the server where Rhapsody is installed, instead of running the application services the default administrator user, set up a separate user to run the Rhapsody service. The two key benefits to this approach are:

- The ability to detect issues that relate to the operating system and Rhapsody interactions and for configuration of operating system and environment settings specific to Rhapsody. For example, setting the maximum number of file handles that can be assigned to the Rhapsody process.
- Ensure that resources assigned to the Rhapsody execution user are not shared with other processes that are not related to the Rhapsody Engine on the server system.

The service account should have access to the requisite network resources and the password policy for the account should be set to never expire.

Account Access to Rhapsody IDE and Management Console

Avoid re-using the default administrator account for multiple users as this makes auditing and tracking of user actions difficult. Ensure that a new account is created for access (either via LDAP or the inbuilt Rhapsody user management) with appropriate roles assigned for each new user. This will enable:

- Restriction of access to Rhapsody IDE features or Management Console functions by role.
- Ability to easily disable a user’s access when required.
- Ability to determine the user responsible for an action based on their user ID (for example, who checked in a given change to Rhapsody IDE).

Cleaning Up Archived Messages

- Schedule routine operations of the Archive Cleanup.
- Schedule the Error Queue Defragmentation process.

Backups

- Consider when to schedule backups – a message backup will pause routes for a short period of time to back up active files.
- Configure an infrequent full backup to the same directory as a frequent incremental; for example, a weekly full backup and a daily incremental backup.
- Each full backup provides a fresh starting point for further incremental backups, rather than having a never-ending chain.

Backups are not deleted by Rhapsody. Refer to Archive Cleanup for details.

Notification Schemes

How notification schemes in the Management Console are configured varies highly from customer to customer depending on their size, priorities of different interfaces, and the number of resources in the team administering Rhapsody. At a minimum, the following notification functionality should be configured and tested for every environment:

- The configuration is set up to send notifications to a default administrative user email account or distribution list name.
- Thresholds for warnings and alerts (for example, large queues, communication point shut down, etc.) have been configured inline with the minimum solution/customer requirements.
- The notification event history is included in the scheduled Archive Cleanup configuration as this can build up over time and take up a considerable amount of disk space.

Security Best Practices

Security Best Practices

Rhapsody's security best practice guidelines are provided with the intention of helping you to develop your own best practices for designing and implementing secure Rhapsody configurations.

Encrypted Rhapsody Variables

Encrypted Rhapsody variables provide the best security in Rhapsody. For sensitive values, using encrypted Rhapsody variables are the best practice. Additionally, RLC encryption should *always* be used.

Encrypting Backups

Ensure you always encrypt a backup archive as it contains the passphrase store.

Virtualization Best Practices

> **Warning**
>
> Do not over-commit your virtualized environment, whether it is in memory or CPU resources. As Rhapsody is an enterprise-level Java application, performance will be severely impacted if the virtual machine hosting Rhapsody does not have sufficient dedicated resources at all times.
>
> Performance profiling should always be conducted with production-level loads prior to deploying Rhapsody configurations to a production environment in order to allocate adequate resources. This is even more crucial when hosting Rhapsody in a virtualized environment.

If you intend to set up Rhapsody on a virtual machine, you must consider the following configuration recommendations in order to optimize performance:

- Do not assign more virtual CPUs (vCPUs) to one virtual machine than you have physical CPUs available.
- Determine the optimal number of vCPUs by testing the performance of virtual machines with different numbers of vCPUs using the expected Rhapsody load.
- Do not over-commit CPUs to your virtual machine.
- Ensure CPU ready time metric is low (below 20 percent).
- Allocate at least enough RAM to cover the JVM memory allocation.

- Do not over-commit JVM memory.
- Ensure you have enough memory on the physical machine to accommodate the active memory of all virtual machines.
- Host cluster settings.
- Prevent memory ballooning and memory resource contention.
- The number of Garbage Collection threads should be less than or equal to the number of vCPUs.
- General virtualization best practices.

Note

The information in this section generally applies to VMware. For specific details on running applications on virtual machines, refer to the virtual machine applications' documentation. General literature on virtual machines is available on the internet.

This section is intended for the VMware and system administrators in charge of managing the Rhapsody server.

Do not over-assign virtual CPUs (vCPUs)

This best practice is dictated by the physical resource constraint on your host because a vCPU is one core in your physical processors. Therefore, the maximum number of vCPUs that can be specified to a virtual machine is equal to the number of cores on the host. Assigning six virtual vCPUs to your virtual machine when only four physical CPUs are available does not increase performance; you are still limited to the performance that the four physical CPUs are able to provide. If you have multiple virtual machines on one host, you can have more total vCPUs allocated across all virtual machines than you have physical CPUs because the physical machine allocates virtual machine processes

to any idle physical CPU. Allocating more vCPUs in this case reduces the physical CPU idle time. However, allocating significantly more virtual CPUs than physical CPUs (roughly more than four or five vCPUs per pCPU) can increase the load on the physical CPUs to the point that they are all running at a high percentage (90-100%). This reduces performance across all virtual machines, including the one running Rhapsody.

Determine the optimal number of vCPUs

The optimal vCPU configuration for your virtual machine is dependent upon the expected Rhapsody load. We recommend at least two vCPUs be allocated, but for larger Rhapsody loads, more vCPUs may give better performance. In the past problems have been identified that were resolved with additional processors. However, allocating more vCPUs to a Rhapsody configuration with a low load is not guaranteed to increase performance and may actually reduce performance, as described in the following best practice. If the CPU usage value for your virtual machine is consistently above or about 80%, then the load is too high for the current CPU configuration to handle (since it would struggle to handle bursts of messages). You should therefore assign additional CPUs.

Do not over-commit CPUs

Assigning more CPUs than are required to your virtual machine gives you no added performance benefits. Running a small Rhapsody load that could be handled easily by two vCPUs on a virtual machine with four vCPUs means that the virtual machine is asking the physical device for four physical CPU cores. The virtual machine will have to wait longer for four physical CPU cores to become available than for two to become available. This increases the wait time for each multi-threaded action the virtual machine tries to do and so overall, the performance will drop. If the exact workload is not known, start with a smaller number of vCPUs and increase later as necessary.

Ensure CPU ready time metric is low (below 20 percent)

CPU ready time is the duration of the time spent by the virtual machine being ready to run against the physical CPUs when none are available, in other words the time spent waiting for an available core to perform the work. If your CPU Ready metric is high, this means your virtual machine is finding it hard to get physical resources. This could be due to several reasons, for example, that you over-committed your physical CPUs. In order to mitigate this scenario, either the CPU resources allocated to the virtual machine need to be reduced or additional hardware resources need to be added to the cluster.

Allocate at least enough RAM to cover the JVM memory allocation

The JVM has a memory heap that is accessed frequently and so should be present in physical memory. Not allocating enough memory to your virtual machine to cover the

heap space will result in frequent page swapping that decreases performance. To be confident that you have enough RAM, only allocate 50% of the memory to the Java Heap. JVM memory allocation can be significantly larger than the heap, especially on 64-bit operating systems, when hyperthreading is a feature on the physical CPUs. With hyperthreading, allocating 50% of the virtual machine's RAM to the Java heap can equate to the JVM taking 70% of the virtual machine's RAM. Refer to Allocating Memory to Rhapsody for details.

Do not over-commit JVM memory

JVM memory is an active space that is constantly being accessed and garbage collected which requires the memory to be available all the time. Assigning more memory than required, specifically more memory than virtually allocated or physically present, may lead to memory ballooning or swapping, reducing performance.

Ensure you have enough memory on the physical machine

If the physical memory is too small to assign specific memory space to the active memory of all virtual machines, then the virtual machines experience ballooning or swapping which decreases performance on these machines. If the physical machine has less than 6% free memory, this indicates it is struggling to meet all the memory requirements.

Host cluster settings

It is a VMware best practice to load balance the virtual machines in your vCenter environment by grouping your hosts into a cluster and enabling the Dynamic Resource Scheduler (DRS). DRS will determine both the best host for a virtual machine to run on at its boot time, and will move virtual machines with vMotion between the hosts to allow additional servers to run. It is recommended to set DRS to automatic mode and the migration threshold to the second level (one level removed from the most conservative setting). This initial setting is recommended for most instances, but it can be changed based on the load and performance in your environment. vMotion can be manually executed on a running or powered off Rhapsody virtual machine when the VMware administrator deems it necessary. This should be avoided during periods of long-running garbage collections. If you are installing Rhapsody in an existing VMware environment, follow the guidance of your local VMware administrator for the best setting for your virtualization usage. Optionally, for additional control over migrating virtual machines, resource pools can be configured. Refer to VMware documentation for all of the available options and settings for resource pools.

Prevent memory ballooning and memory resource contention

Memory ballooning allows the virtualization host to redistribute memory between active virtualized environments sharing the same physical memory pool. If you have memory

ballooning enabled, then it is possible that another virtual machine could request more memory which would result in memory being re-allocated away from the environment hosting Rhapsody. This ultimately results from over-committing memory to your virtual machines.

Due to memory management and garbage collection techniques employed by Java, Java application performance is sensitive to changes in available memory. Virtual machine ballooning can lead to the reallocation of memory from the system hosting Rhapsody to other system memory that has not yet been committed to the Java Heap.

The operating system may then page memory out to disk/swap space, usually according to an LRU (least recently used) algorithm, because there is insufficient physical memory for its needs. If ballooning occurs, this would likely trigger exhaustion of physical memory on the system hosting Rhapsody, which would then lead to memory in the old/tenured space being swapped out to disk. A full garbage collection would then need to page this back into memory in order to perform garbage collection. The resulting extra I/O makes a full garbage collection slower by an order of magnitude. Since locking techniques in the garbage collector are optimized for physical memory, this can cause a long garbage collection pause in application processing in the Rhapsody engine.

Simply setting a memory reservation on the virtual machine equal to the entire allocated memory of the virtual machine can mitigate this issue because ballooning will never be used if all of the virtual machine's memory is reserved. The benefit of this method is that all the memory the virtual machine needs is reserved and therefore the hypervisor does not consider that virtual machine for ballooning. This method continues to allow the hypervisor to use memory ballooning for the other running virtual machines if needed.

Note

Your hypervisor vendor may refer to memory ballooning using a different term, such as dynamic memory allocation.

The number of Garbage Collection threads should not be more than the number of vCPUs

If you have more garbage collection threads than virtual CPUs, you will experience context switching between your garbage collection threads. Some garbage collection threads will be held up waiting for other threads to complete. Since context switching

incurs a time penalty, this slows down the time to complete a garbage collection run. To set the number of garbage collection threads on the JVM use -XX:ParallelGCThreads=<n>. Generally on a virtual machine with only one JVM, this parameter is automatically configured, but it can be explicitly specified to finer tune your environment. In the case where there are multiple JVMs on the virtual machine, you must adjust this parameter on all of your JVMs to optimize CPU usage for each one. If a full garbage collection takes more than five minutes, Rhapsody assumes that the garbage collection has hung and thus restarts the Rhapsody engine. When a full garbage collection is underway, all other threads are paused, in effect pausing Rhapsody and stopping any messages from being processed. The duration of garbage collection can be reduced significantly by increasing the vCPUs to the virtual machine and adjusting the JVM -XX:ParallelGCThreads=<n> parameter accordingly. This is especially useful in high throughput environments and ones where the JVM is allocated larger amounts of RAM. It is important to remember that CPU processing in a virtualized environment only occurs when the number of allocated cores (vCPUs) to the virtual machine are available simultaneously. So the larger the number of vCPUs allocated to the virtual machine, the longer the virtual machine may have to wait to process on the physical hardware. It should be noted that when hyper-threading is available on the physical CPU that the number of cores does not equal the number of threads available, but when doing computing resource allocation to the JVM it is advised to consider the number of threads equal to the count of vCPUs in the virtual machine. You should conduct performance testing to determine what the right settings are for your environment. This effect can be further mitigated by configuring CPU reservations and shares.

General virtualization best practices

- Only run on versions of hypervisor products that are actively supported.
- Where your hypervisor provides guest-agent tools for managing and monitoring the guest virtual machines, ensure these are installed.
- Set all of your hypervisor hosts to use the same internal NTP server for time synchronization.
- Guest virtual machines should be configured to use the same NTP source, or alternatively, have guest-agent tools synchronize the time with the host.
- Configure DNS servers for short and fully qualified domain names for your hypervisor hosts.
- Configure DNS servers for your Rhapsody virtual machines for both short and fully qualified domain names.

- When allocating virtual machine memory resources, allocate enough for the operating system and all running applications, including the Rhapsody JVM.
- When using virtualized disks on supported physical architecture for the Rhapsody datastore, provision them as thick provision with the eager zero option for maximum performance.
- Limit the use of snapshots in production as they can negatively impact disk I/O performance.
- Utilize a system monitoring product to proactively monitor the health of your hosts, storage, virtual machines, and applications.
- Keep all drivers, firmware, and BIOS up to date on your hypervisor hosts.
- Test and apply all hypervisor software patches in a timely manner.
- Unless you plan to move a virtual machine to older versions of hypervisor hosts, always use the highest version of virtual hardware compatible with all of your hosts and keep guest-agent installed with the version applicable to your firmware level. More modern CPU architectures typically have features that are of benefit to Rhapsody processing workloads (for example, accelerators for compression and encryption).

Refer to Best practices for running Java in a virtual machine (1008480) for additional information.

Maintenance Guidelines

To prevent the performance of your Rhapsody engine from degrading over time, it is recommended you observe certain best practices to maintain the engine. Important factors that determine engine health are listed in this topic.

Archive Cleanup

Archive Cleanup removes 'dead' messages that are older than the configured archive period. A message is considered 'live' when it is traversing through Rhapsody, and dead when it has left Rhapsody. A common cause of disk space issues is having a large number of live messages. Any messages that are in the Error Queue, the Hold Queue, communication point queues, route queues, or message collectors are considered live and therefore are not be eligible for archive clean-up. Due to the structure of the Rhapsody datastore, having even a single live message can render other messages, that have been received by Rhapsody at approximately the same time, ineligible for clean-up as well. Consequently, a large number of live messages can make many other messages and their associated files (body files, metadata files, indexes) ineligible for clean-up.

To keep disk space utilization low, it is recommended you minimize the number of live messages within Rhapsody. Ensure you do not have a large number of messages in the Error Queue, Hold Queue, route queues, or communication point queues. For example, if you need to hold messages for extended periods of time (measured in days) due to the unavailability of the recipient system, it is recommended you write those messages to disk using a Directory communication point rather than hold them within Rhapsody.

Backups

It is recommended you schedule regular backups of your configuration and administration areas, so that you can restore your configuration in the event it is corrupted. The backup frequency should depend on how often you make configuration changes. For example, if your configuration only changes once a month, there is little point in backing up your configuration on a daily basis.

In most cases, a message store backup is not necessary. If your message store becomes corrupted, restoring from a backup of the message store could result in old messages being replayed. If you do need to configure backups of the message store, then it is important to be aware of the potential performance impact. To ensure the integrity of a backup, it is necessary to pause sections of the live datastore during the backup process. If you have large live queues or a large configuration, the pause duration could be large. When queues are paused, messages held in those queues are not processed and overall engine throughput could drop. The size of the message store will impact the total time taken to perform a message store backup. Consequently, it is

recommended you only back up the configuration and administration areas unless you have a specific requirement to back up the entire message store.

Error Queue and Hold Queue

Any messages in the Error Queue or Hold Queue are considered live, and are not eligible for archive clean-up – which increases disk space requirements. In Rhapsody versions prior to Rhapsody 6.2.1, messages in the Error Queue and Hold Queue are also validated during start-up, resulting in long startup times if the number of messages in these queues are high.

It is therefore recommended you address messages in the Error Queue on a daily basis, determining why a message has been sent to the Error Queue and resolving the root cause. The Error Queue is designed to catch unexpected exceptions – its message count should be kept to a minimum and should not be used as a repository for messages with known issues.

The Hold Queue should not be used in production environments.

Communication Point and Route Queues

Large communication point and route queues can have a number of detrimental effects. Messages in these queues are considered live and hence not eligible for cleanup. These messages are validated on start-up and can cause long start-up times if the number of messages in these queues is large. If a backup of the message store is attempted, then communication point and route queues are paused as required by the backup process. The larger the queue, the longer the pause time. References to messages in a route queue are held in memory. Having an abnormally large route queue could cause higher heap memory usage. Consequently, it is recommended that you ensure the number of messages in communication point queues and route queues is low. Typically these queues build up when routes or communication points are kept in the stopped state while the feeders are still running. The same applies to messages held in message collectors. Collection time periods should be short – in seconds or minutes rather than hours or days.

Lookup Tables Failure List

By default, Rhapsody logs lookup table failures and keeps a list of all unique failures. It is recommended you monitor the failure list and update the lookup tables as necessary. If this is not done, the failure list can grow very large over time and have a performance impact on your engine. When the lookup failure count is very high, attempting to view the Monitoring Lookup Tables page in the Management Console could result in high heap memory usage and the Management Console page failing to display. Similarly, attempting to view the lookup tables from Rhapsody IDE will also lead to high heap memory usage and could potentially lead to Rhapsody IDE hangs.

Monitoring Memory Usage

The heap memory requirement is dependent on a number of factors such as message load, the size of the messages, the size of the configuration, and the operations performed. When new message feeds are added or when the configuration is modified, it is recommended you check the impact on memory. Ideally, heap memory usage should fluctuate between 40% and 80% of the maximum heap size. This gives enough space for occasional spikes in memory usage. When a spike occurs, determine what is causing the memory spike. Causes typically include full-text searches, attempting to process large messages, and bursts of incoming messages. Memory spikes may trigger warnings about high heap memory usage. These can be ignored as long as the memory is reclaimed shortly afterward by the garbage collection (GC) process. It only becomes a matter of concern if the memory usage goes up above 80% and stays above that threshold for an extended period of time.

A typical misconception is that more heap memory allocation is better. As heap sizes grow, the chances of encountering long GC pauses increase. This is because, as the heap becomes larger, the frequency of full garbage collection cycles decreases. When a full GC eventually gets triggered, it needs to clean up a large number of objects. Since a full GC is a 'stop the world' process, the engine is unresponsive during that process. For applications such as Rhapsody that are sensitive to response times, it is better to have frequent short full GCs rather than the occasional long full GC.

Capacity Review When Message Load Increases

The resource requirements (memory, CPU, disk I/O, disk space) can change when new message feeds are added, when the message load changes, or when configuration changes are made. For this reason, it is recommended you perform a capacity review on a regular basis to prevent potential resourcing problems in the future.

Service Pack Updates

Rhapsody releases regular service packs to address the bugs that have been identified in the field. Most service packs contain upwards of 50 bug fixes. Rhapsody as an organization goes to some length to ensure that service packs do not change existing behavior (except, of course, addressing bugs) so that the testing effort required prior to applying service pack updates is minimal. It is strongly recommended you keep up-to-date with service packs to minimize the number of bugs in your production engines. Service pack releases are announced in the Rhapsody forums.

Procedures for Restarting the Engine

When a Rhapsody engine needs to be shut down for maintenance, keep the number of messages in-flight within Rhapsody to a minimum. This ensures the engine start-up is

fast and only a small number of transactions need rolling back. The recommended shutdown procedure is as follows:

1. Shut down all inbound communication points.
2. Give enough time for messages in routes to flow to outbound communication points and then shut down all routes.
3. Give enough time for messages in outbound communication points to leave Rhapsody and then shut down outbound communication points.

An exception to this procedure is a communication point in Bidirectional or In->Out mode. In this case, you cannot shut down the inbound direction without shutting down the outbound direction as well. Therefore, ensure there are no messages in route queues before shutting down the engine.

Monitoring the Size of a Configuration for a Single Engine

There is a limit to the size of a configuration (the number of communication points, routes, and filters) that a single Rhapsody engine can handle in a performant manner. The recommended maximum size of a configuration is approximately 2,000 communication points. This number is based on tests performed using TCP Client and TCP Server communication points. Users typically begin to notice slowness on Management Console pages and Rhapsody IDE check-ins before seeing any performance degradation in message processing. For configurations with more resource-intensive communication points, the maximum size will be less (for example, a web service communication point is more resource-intensive than a Timer or Sink communication point, and a Mapper filter is more resource-intensive than a No-operation filter).

Notifications to Handle Issues Proactively

Rhapsody provides the option of configuring notifications. It is important to configure alerts on items that you care about. While Rhapsody ships with default threshold values for these notifications, it is recommended you modify the threshold values to suit your environment. For example, if a message spends more than a certain number of seconds in a filter, you could configure it to get a notification. If you have a database filter that has a query that is likely to take a long time to execute, you should tweak the thresholds accordingly so that you do not receive unnecessary alerts (however, it is recommended you use the Database communication point for such cases). Notifications enable you proactively fix issues before they become serious.

Startup States

If there are hard dependencies in your configuration, you should use startup states to ensure that the communication points and routes that are needed for other components to work are started before the components that depend on them.

If there are no hard dependencies, then general best practice is to observe the following sequence:

1. Start output mode communication points.
2. Start routes and dynamic routers.
3. Start input mode communication points.

Upgrading Checklist

The following table outlines the steps Rhapsody as an organization recommends for updates and upgrades, presented in one checklist, with indications of the steps that are specific to the upgrade process.

> **Note**
>
> - The checklist assumes a multi-environment upgrade process; the steps in the checklist must be performed in one or more non-production environments initially and then in the production upgrade.
> - Non-production environments should match the production environment as closely as possible while not interfering with the production environment in any way.
> - We strongly recommend at least two non-production environments. This ensures that there is an opportunity to upgrade, revise and verify the upgrade process.

Complete the following checklist for each environment being affected by product changes:

1. Before Upgrade	2. Upgrade	3. Functional Testing	4. Non-functional Testing	5. Final Steps
Review relevant manual upgrade steps in the	Follow installation steps as	Re-run the test message validation set to ensure the	Observe message throughput on routes and	Document outcomes, changes and

flowchart in Upgrade Caveats. ☐ Done ☐ N/A (update only)	defined for your version. ☐ Done	same results (output messages) as the pre-upgrade test. ☐ Done	interfaces and compare with pre-upgrade statistics. ☐ Done	learnings for future upgrades. ☐ Done
Engage PSG? ☐ Yes ☐ No	Follow manual processes required, as reviewed previously. ☐ Done ☐ N/A (update only)	End-to-end testing: do end-results in downstream systems appear as expected? ☐ Yes ☐ No		
Upgrade plan documented, including responsibilities and timing (change windows etc.), corporate policy and rollback plan. ☐ Done		Ensure that all stopped components were in the stopped state previously. ☐ Done		
Current production state documented, including environmental info (usernames, datastore location etc.), PDF of configuration, stopped				

components, Error Queue count. ☐ Done				
For a production upgrade, backup configuration. ☐ Done ☐ N/A (non-prod.)				
For production upgrade, backup datastore. ☐ Done ☐ N/A (non-prod.)				
For a non-production upgrade, import production configuration (changing environment variables as required). ☐ Done ☐ N/A (prod.)				
Run a test message validation set with production data. ☐ Done				
Record typical message throughput on				

routes and interfaces. ☐ Done				